



Entornos virtuales y dependencias

Objetivos

- Entender qué es un entorno virtual y por qué usarlo en cada proyecto.
- Crear, activar y desactivar un `.venv` con el módulo `venv`.
- Instalar dependencias con `pip` y documentarlas en `requirements.txt`.
- Evitar subir el entorno virtual y los secretos a Git (`.gitignore`).
- Guardar la API key en un fichero `.env` local y cargarlo con `python-dotenv`.
- Reproducir el proyecto en [ejemplos_proyecto_entornos_virtuales/](#) (`gemini_hola_mundo.py` o `gemini_hola_mundo.ipynb`).
- Conectar el **kernel** del notebook al Python del `.venv` de esa misma carpeta.

API key (proyecto Gemini): [Google AI Studio](#) · **Docs:** [Gemini API](#)

1) ¿Qué es un entorno virtual?

Un **entorno virtual** en Python es un entorno aislado que permite ejecutar proyectos con sus propias dependencias y bibliotecas, independientemente de lo instalado en el **Python global** del sistema.

Es especialmente útil cuando gestionas varios proyectos: puedes instalar **distintas versiones** de la misma librería en entornos distintos sin que se interfieran. Así el proyecto es más **reproducible** y reduces conflictos entre requisitos.

En la práctica, el entorno suele ser una carpeta (por convención `.venv`) con su propio `python` y su propio `pip`.

En notebooks (Jupyter, VS Code, Cursor u otro IDE): el código no usa “el Python del sistema” por arte de magia, sino el **kernel** que elijas. Si instalaste paquetes en `.venv`, el kernel debe apuntar a ese intérprete.

2) `venv`: crear, activar y desactivar

`venv` viene en la biblioteca estándar desde **Python 3.3**. Es la opción recomendada en este bootcamp.

Crear el entorno

En la carpeta del proyecto:

```
python -m venv .venv
```

En macOS/Linux, si el comando es **python3**:

```
python3 -m venv .venv
```

En algunas distribuciones Linux puede hacer falta instalar antes el paquete del sistema:

```
sudo apt-get install python3-venv # Debian/Ubuntu
sudo dnf install python3-venv     # Fedora/RHEL
```

Activar el entorno

Tras activar, verás (**.venv**) al inicio del prompt. Lo que instales con **pip** quedará **solo** en ese entorno.

Sistema	Comando
Windows (PowerShell)	<code>.\.venv\Scripts\Activate.ps1</code>
Windows (cmd)	<code>.venv\Scripts\activate.bat</code>
Windows (Git Bash)	<code>source .venv/Scripts/activate</code>
macOS / Linux	<code>source .venv/bin/activate</code>

Comprobar:

```
python -c "import sys; print(sys.executable)"
```

Desactivar

```
deactivate
```

Vuelves al Python global del sistema.

PowerShell: error al activar

Si aparece un mensaje del tipo *"la ejecución de scripts está deshabilitada"* al lanzar **Activate.ps1**:

1. **Alternativa rápida:** usa **cmd** o **Git Bash** con `activate.bat` / `source .venv/Scripts/activate`.
2. **Solo esta sesión de PowerShell:**

```
Set-ExecutionPolicy Unrestricted -Scope Process
```

3. Recomendado (usuario actual):

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

`RemoteSigned` permite scripts locales; los descargados suelen requerir firma.

3) Instalar paquetes con `pip`

Con el entorno **activado**:

```
python -m pip install -U pip
python -m pip install nombre_paquete
```

Ejemplo genérico:

```
pip install numpy pandas scikit-learn
```

Los paquetes se instalan **solo** en el entorno virtual activo, no en el Python global.

4) Git: no subir el entorno virtual

Con Git, **no subas** la carpeta `.venv` al repositorio: es pesada y depende del sistema operativo.

En `.gitignore` del proyecto, incluye al menos:

```
# Entornos virtuales
venv/
env/
.venv/
ENV/

# Secretos (fichero .env en la raíz del proyecto)
.env

# Python
__pycache__/
*.pyc
```

¿Por qué ignorar `.venv`?

- Ocupa mucho espacio.
- Es específica de cada máquina / SO.
- Puede generar conflictos entre colaboradores.

Qué sí compartir: `requirements.txt`, `.env.example` (plantilla sin claves reales) y el código fuente.

Qué no compartir: el fichero `.env` con tus claves.

5) Variables de entorno con fichero `.env`

Las **variables de entorno** son pares `NOMBRE=valor` que el sistema (o tu proceso Python) puede leer con `os.getenv("NOMBRE")`.

Para proyectos con API keys conviene:

1. **No** pegar la clave en el código ni en el notebook.
2. Guardarla en un fichero `.env` en la carpeta del proyecto (solo en tu máquina).
3. Añadir `.env` al `.gitignore`.
4. Subir al repo un `.env.example` sin secretos, para que el equipo sepa qué variables hacen falta.

Crear el fichero `.env`

En `ejemplos_proyecto_entornos_virtuales/`, copia la plantilla y edita tu clave:

```
# Windows (PowerShell)
copy .env.example .env

# macOS / Linux / Git Bash
cp .env.example .env
```

Contenido de `.env.example` (sí va al repositorio):

```
GEMINI_API_KEY=tu_clave_de_google_ai_studio
```

Contenido de `.env` (no va al repositorio; sustituye por tu clave real):

```
GEMINI_API_KEY=AIza...tu_clave_real...
```

Reglas útiles:

- Una variable por línea: `NOMBRE=valor`.
- Sin comillas obligatorias (puedes usarlas si el valor tiene espacios).
- No dejes espacios alrededor del `=` si quieres evitar sorpresas (`GEMINI_API_KEY=abc`).

Cargar `.env` en Python con `python-dotenv`

Instálalo en el `.venv` (ya está en `requirements.txt` del proyecto):

```
pip install python-dotenv
```

En el script o en la celda de configuración:

```
import os
from dotenv import load_dotenv

load_dotenv() # lee .env en la carpeta actual (o padre) y rellena os.environ

clave = os.getenv("GEMINI_API_KEY")
```

`load_dotenv()` **no sustituye** variables que ya existan en el sistema; respeta lo que exportaste antes en la terminal. Si quieres forzar el `.env`, usa `load_dotenv(override=True)` (solo en desarrollo local).

El SDK de Gemini usa `GEMINI_API_KEY` automáticamente tras `load_dotenv()`.

Orden recomendado de configuración

1. Crear `.env` desde `.env.example`.
2. Ejecutar el proyecto (script o notebook): `load_dotenv()` carga la clave.
3. Si no hay `.env` ni variable en el sistema, el proyecto puede pedir la clave con `getpass` (respaldo en el ejemplo).

Alternativa sin `python-dotenv`

Exportar en la terminal antes de ejecutar:

```
# PowerShell
$env:GEMINI_API_KEY = "tu_clave"

# Git Bash / macOS / Linux
export GEMINI_API_KEY="tu_clave"
```

Útil en servidores; en desarrollo local el `.env` suele ser más cómodo.

6) `requirements.txt`

Documenta las dependencias del proyecto para que otra persona (u otro equipo) pueda reproducir el entorno.

Generar (con `.venv` activo y paquetes ya instalados):

```
pip freeze > requirements.txt
```

Instalar en un entorno nuevo:

```
pip install -r requirements.txt
```

Fijar versiones (recomendable cuando el proyecto crece):

```
# Python 3.10+  
google-genai>=1.0.0
```

O con versiones exactas, por ejemplo `numpy==1.21.0`, para evitar sorpresas al actualizar.

En nuestro proyecto de ejemplo:

```
google-genai>=1.0.0  
python-dotenv>=1.0.0
```

7) Python concreto y otras herramientas (referencia)

Varias versiones de Python

Si tienes varias versiones instaladas:

```
# Windows  
py -3.10 -m venv .venv  
  
# macOS / Linux  
python3.10 -m venv .venv
```

Con **pyenv** (macOS/Linux): `pyenv install 3.10.x`, `pyenv local 3.10.x`, luego `python -m venv .venv`.

Comprueba dentro del entorno activo:

```
python --version
```

Alternativas a **venv** (panorama)

Herramienta	Cuándo tiene sentido
venv	Proyectos Python estándar; incluido en Python 3.3+ (recomendado aquí).
Conda	Ciencia de datos, stacks con dependencias no-Python; Anaconda/Miniconda.
virtualenv	Proyectos legacy; la mayoría de ventajas ya están en venv .

En esta unidad usamos **venv** + **pip** + **requirements.txt**.

8) Estructura del proyecto de ejemplo

Todo lo reproducible está en **ejemplos_proyecto_entornos_virtuales/** (abre solo esta carpeta en el editor):

```

01_Teoria/
├── 01_Entornos_virtuales_y_dependencias.md    ← este documento
├── 04_Ejemplos_Gemini_API_con_Python.ipynb    ← ejemplos ampliados (JSON,
streaming, chat; opcional)
├── ejemplos_proyecto_entornos_virtuales/      ← proyecto de la sesión
│   ├── .venv/                                ← python -m venv .venv (lo creas tú)
│   ├── requirements.txt
│   ├── .gitignore
│   ├── .env.example                          ← plantilla (sin clave real)
│   ├── .env                                  ← tus secretos (lo creas tú; no en git)
│   ├── gemini_hola_mundo.py                  ← script: .env + hola mundo
│   └── gemini_hola_mundo.ipynb               ← notebook: mismos pasos 0, 1 y 2 en celdas

```

gemini_hola_mundo.py y **gemini_hola_mundo.ipynb** comparten el mismo código; el notebook lo divide en celdas (instalación, API key, primera llamada).

Tutorial paso a paso

Sigue los pasos **en orden**. Incluye el contenido de cada fichero y los comandos de terminal.

Paso 1 — Ir a la carpeta del proyecto

```
cd ruta/a/01_Teoria/ejemplos_proyecto_entornos_virtuales
```

Si clonas el material del bootcamp, los ficheros ya están; solo crea **.venv** e instala dependencias.

Paso 2 — Crear los ficheros (si partes de cero)

requirements.txt

```
google-genai>=1.0.0  
python-dotenv>=1.0.0
```

.gitignore

```
.venv/  
.env  
__pycache__/  
*.pyc
```

.env.example

```
GEMINI_API_KEY=tu_clave_de_google_ai_studio
```

Cópialo a `.env` y sustituye por tu clave de [Google AI Studio](#).

gemini_hola_mundo.py

```
# Ejemplo de script para ejecutar nuestro proyecto  
import os  
import getpass  
  
from dotenv import load_dotenv  
from google import genai  
  
load_dotenv() # carga variables desde .env (si existe)  
  
if not os.getenv("GEMINI_API_KEY"):  
    os.environ["GEMINI_API_KEY"] = getpass.getpass(  
        "Pega aquí tu GEMINI_API_KEY (input oculto): "  
    )  
  
print("GEMINI_API_KEY configurada:", "sí" if os.getenv("GEMINI_API_KEY") else  
      "no")  
  
client = genai.Client()  
MODEL = "gemini-3-flash-preview"  
  
response = client.models.generate_content(  
    model=MODEL,  
    contents="Resume en 3 frases qué es la IA generativa.",  
)  
  
print(response.text)
```


Si usas el repo del bootcamp, los ficheros ya están en [ejemplo_proyecto_entornos_virtuales/](#) (incluido `gemini_hola_mundo.ipynb`); crea tu `.env` y pasa al Paso 3.

Paso 3 — Crear el entorno virtual

Desde `ejemplo_proyecto_entornos_virtuales/`:

```
python -m venv .venv
```

(o `python3 -m venv .venv`)

Paso 4 — Activar el entorno virtual

Usa la tabla del apartado **2)**. Comprueba que la ruta de `sys.executable` contiene `ejemplo_proyecto_entornos_virtuales` y `.venv`.

Paso 5 — Instalar dependencias

Con `(.venv)` activo:

```
python -m pip install -U pip
python -m pip install -r requirements.txt
```

En el notebook (kernel del `.venv`), la celda de instalación puede ser:

```
# %pip install -U google-genai python-dotenv
```

Paso 5b — Crear tu fichero `.env`

Desde `ejemplo_proyecto_entornos_virtuales/`:

```
cp .env.example .env
```

Edita `.env` y pega tu `GEMINI_API_KEY`. **No** subas `.env` a Git (ya está en `.gitignore`).

Paso 6 — Ejecutar el proyecto

Opción A — Notebook (recomendada en clase)

1. Abre [gemini_hola_mundo.ipynb](#) en esta misma carpeta.
2. **Select Kernel** → Python de [ejemplo_proyecto_entornos_virtuales/.venv](#) (Paso 7).
3. Ejecuta las celdas **0** (instalación), **1** (API key) y **2** (primera llamada).

Opción B — Script en terminal

```
python gemini_hola_mundo.py
```

1. Carga la clave desde [.env](#) con `load_dotenv()` (o la pide con `getpass` si falta).
2. Confirma que `GEMINI_API_KEY` está configurada.
3. Imprime la respuesta de Gemini.

Material ampliado (JSON, streaming, chat): [04_Ejemplos_Gemini_API_con_Python.ipynb](#) en [01_Teoria/](#), con el mismo kernel del [.venv](#).

Paso 7 — Kernel del notebook

VS Code

1. Abre la carpeta [ejemplo_proyecto_entornos_virtuales/](#) (o el repo completo).
2. Abre [gemini_hola_mundo.ipynb](#).
3. **Select Kernel** → intérprete en
...\[ejemplo_proyecto_entornos_virtuales/.venv/Scripts/python.exe](#) (Windows) o
.../[ejemplo_proyecto_entornos_virtuales/.venv/bin/python](#) (macOS/Linux).

```
import sys
print(sys.executable)
```

Jupyter clásico (opcional), con [.venv](#) activo en [ejemplo_proyecto_entornos_virtuales/](#):

```
python -m pip install ipykernel
python -m ipykernel install --user --name=entornos-gemini --display-name="Python
(entornos gemini)"
```

Paso 8 — Resumen de comandos

```
cd 01_Teoria/ejemplo_proyecto_entornos_virtuales

python -m venv .venv
# Activar .venv (según tu SO)

python -m pip install -U pip
```

```
python -m pip install -r requirements.txt

cp .env.example .env
# editar .env con tu GEMINI_API_KEY

python gemini_hola_mundo.py
# o abrir gemini_hola_mundo.ipynb y ejecutar celdas 0 → 1 → 2
```

Checklist

- ☐ Ficheros del proyecto en `ejemplo_proyecto_entornos_virtuales/` (incl. `.env.example`, script y notebook).
- ☐ `.env` creado desde `.env.example` con tu clave (y `.env` en `.gitignore`).
- ☐ `.venv` creado y activado (o kernel del notebook = ese `.venv`).
- ☐ `pip install -r requirements.txt` sin errores (`google-genai`, `python-dotenv`).
- ☐ `load_dotenv()` carga `GEMINI_API_KEY` (o respaldo con `getpass`).
- ☐ `gemini_hola_mundo.ipynb` (celdas 0–2) o `python gemini_hola_mundo.py` muestra texto del modelo.